Verdict Explorer in production: changes by team

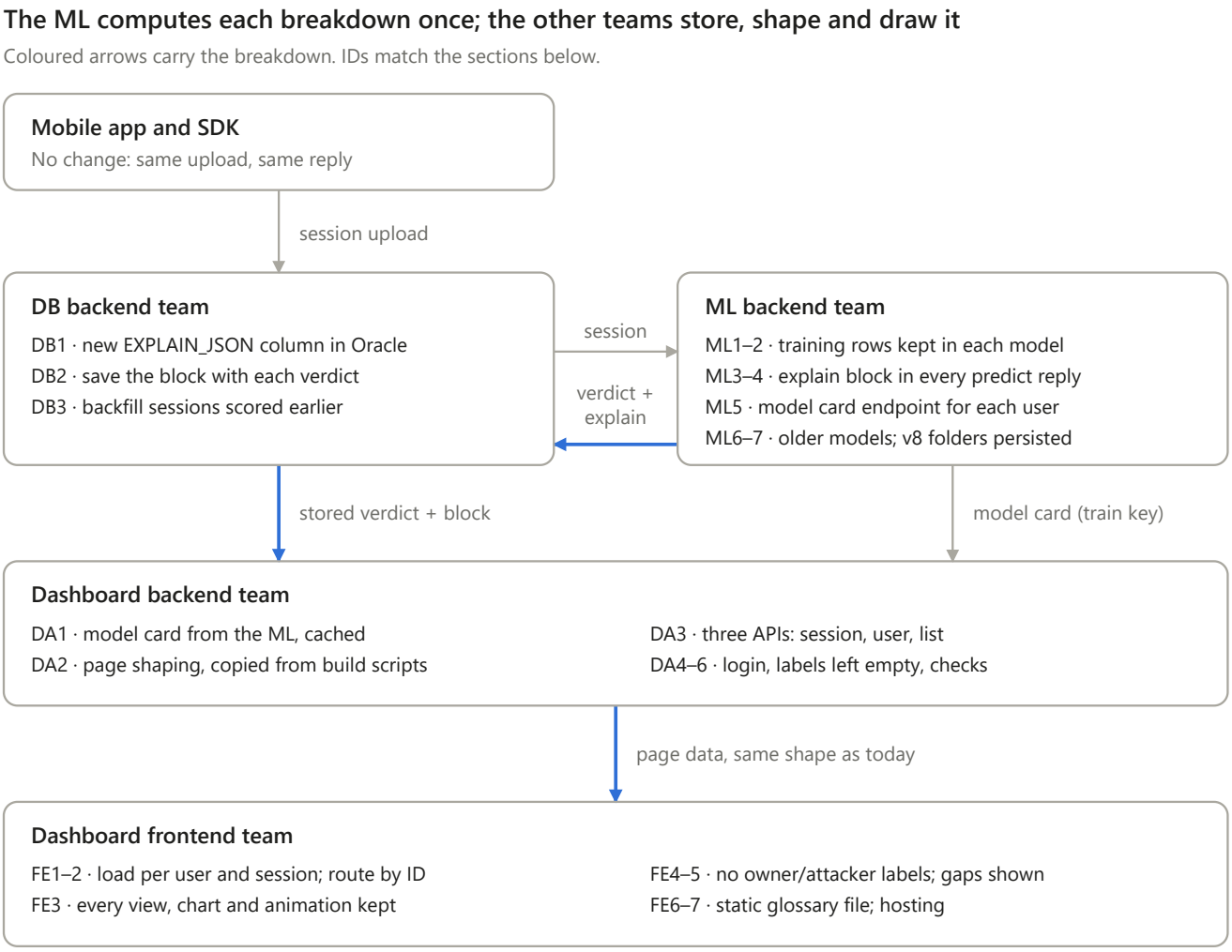
20 Oct 2026 · version 2: with “On the dashboard” boxes · Sri

Today the Verdict Explorer is rebuilt offline from a research copy of the data. These changes let it read every number straight from production: the ML backend computes the explanation in the same request as the verdict, the DB backend stores it, and the dashboard serves and draws it. Verdicts, scores, risk scores and the mobile app do not change.

Contents

1. At a glance
2. Order of work and hand-offs
3. What each change puts on the dashboard
4. ML backend team
5. DB backend team
6. Dashboard backend team
7. Dashboard frontend team
8. What changes, and what production cannot show
9. Open questions to settle before starting

At a glance



who changes what · 4 teams, data flow

The app's upload goes to the DB backend, which asks the ML for the verdict. The verdict and its breakdown come back together, are stored, then shaped by the dashboard backend and drawn by the frontend. The model card goes straight from the ML to the dashboard backend.

Order of work and hand-offs

All four teams can start on day one. The page already has the exact data shape the new APIs will return, so the frontend can build against today's file while the backends are built.

Team	Delivers	Needs from others	Can start
ML backend	ML1–ML7: training rows kept in each model, an `explain` block in every predict reply, `GET /nav/ai/card/<user_id>`, older models filled in, v8 folders on persistent storage	Nothing to build. Switch ML4 on (ML_EXPLAIN=true) only after DB2 is live	Now
DB backend	DB1–DB3: `EXPLAIN_JSON` column, saving the block with each verdict, filling in older sessions	Nothing to build. DB2 must be live before ML4 is switched on, or DETAILS_JSON gets cut short	Now (DB1, DB2); DB3 after ML4 is in production
Dashboard backend	DA1–DA6: three dashboard APIs (user, session, session list) that return the page's data already shaped, plus replay, caching and access	ML5 (model card) and DB2 (stored blocks)	Now, against the ML team's sample replies
Dashboard frontend	FE1–FE7: the page loads from the three APIs, keeps every chart and animation, hides what production cannot know	The API response shape (DA3), which is the shape the page already uses	Now, using `work\dashboard_data_risk.json` as a stand-in

The IDs (ML1, DB2, DA1, FE3 ...) are used in every section below, so a team can point at the exact change it is waiting for.

What each change puts on the dashboard

Every change below now has a coloured **On the dashboard** box: what it makes appear, what the page loses without it, and a screenshot of that part in today's Verdict Explorer. The colours:

Required The page needs it **Optional** Part of a set you can skip

Required, not visible Needed, but nothing new appears **Changes the page** Something is hidden or worded differently

Change	What it puts on the dashboard	Status
ML1	Enrolment ranges, "typical" values, percentiles, mini ranges, fingerprint band	Required
ML2	Nothing by itself: makes ML1's rows and the calibration scores readable	Required, not visible

Change	What it puts on the dashboard	Status
ML3	The maths of every session explanation (used by option A or B)	Required
ML4	Stored breakdown per session; the person page's cross-session views	Optional
ML5	Risk card profile and S-curve, risk-point conversion, ranges, reference sessions	Required
ML6	ML1's parts for users enrolled before the release	Optional
ML7	Nothing new: models survive a container restart	Required, not visible
DB1	Storage for ML4's breakdown	Optional
DB2	Saves ML4's breakdown; keeps DETAILS_JSON intact	Optional
DB3	Breakdowns for sessions scored before the release	Optional
DA1	Brings ML5's model data to the page	Required
DA2	Risk points that add up exactly; checks before showing	Required
DA3	Session page, user context and person page data	Required
DA4	Nothing visible: who may open the pages	Required, not visible
DA5	Owner/attacker labels stay empty	Changes the page
DA6	Nothing visible: checks	Required, not visible
FE1	Every page loads live data	Required
FE2	Links by session ID, pickers, How-page example	Required
FE3	Every existing view and animation stays	Required
FE4	Owner/attacker wording, colours and sections removed	Changes the page
FE5	"Breakdown not available" note	Changes the page
FE6	Definitions page, replay field names	Required
FE7	Opens from the dashboard's session logs	Required

The optional set: ML4, DB1, DB2 and DB3

These four go together. Skip all of them, or do all of them, but never switch ML4 on without DB2. If you skip them, the ML team adds a small explain endpoint and the dashboard backend calls it when a session page opens.

Choice	ML team	DB backend team	Dashboard backend team
A. Store at predict time (as written below)	ML3 + ML4	DB1 + DB2 (DB3 optional)	Reads the stored breakdown
B. Compute when the page opens	ML3 + `POST /nav/ai/explain`	Nothing	Sends the stored session to the explain endpoint; shows the breakdown only when the recomputed score equals the stored one

Whichever you choose, ML1–ML3, ML5 and ML7 are still needed.

Part of the page	No ML4, no option B	Option B	Needs ML4
Verdict, risk score, gauge	Shown	Shown	No
Risk card: 0–100 scale, S-curve, tiles (with ML5)	Shown	Shown	No
Session details, replay, definitions page	Shown	Shown	No
“This session” values, mini ranges, percentiles, fingerprint	Missing	Shown	No, B covers it
Pull/push lists with points, tug of war, waterfall, Why text, exact arithmetic	Missing	Shown	No, B covers it
“Compared against”, the closest reference sessions	Missing	Shown	No, B covers it
“What would flip the verdict”	Missing	Shown	No, B covers it
Sanitizer reason and event counts	Missing	Shown	No, B covers it
Person page: deviation, allowance and ratio in the session list	Missing	Every session re-extracted on each view	**Yes**

Part of the page	No ML4, no option B	Option B	Needs ML4
Person page: time-order chart and S-curve dots for all sessions	Missing	Same	**Yes**
Person page: which behaviour groups drive rejections	Missing	Same	**Yes**
A session's breakdown after a later ML release changes feature extraction	Missing	Wrong or missing	**Yes**

Why A is still the better production choice: the breakdown comes from the very values the verdict used, so it is exact by construction; it survives later ML releases; it is one Oracle read instead of a full re-extraction (about 130–600 ms per view in our earlier timing); and it keeps dashboard browsing off the scoring service. Option B is enough for a demo or occasional single-session views.

Why ML5 cannot be skipped

The model's details are saved only in the model file on the ML server (`/app/saved_models_v8/<user>_ocsvm.joblib`). The dashboard backend only has Oracle, and today it uses none of this: its APIs read logs, users, risk scores, key timings and device details, and the backend calls the ML only for `/predict` and `/train`.

Needed by the page	In Oracle today?
Threshold	Yes, in every session's `DETAILS_JSON`
Training sessions' raw data	Yes (`NAV_SESSIONS`), but the 50 values need the ML's feature code
α, ρ, support vectors, scaler, calibration scores	No, only inside the model file

Reading the model file directly would mean mounting the ML's folder and matching its scikit-learn version, which couples the two services. The ML5 endpoint, or the ML pushing the same data to Oracle at enrolment, is the clean way. Today's Verdict Explorer uses all of this data too; it was copied out of the model files offline on the research PC.

ML backend team

The ML service computes the explanation in the same request as the verdict, and serves each user's model to the dashboard. Repository: `docker-compose` (the ML service). Paths below are from its root. The verdict, score, threshold and risk score stay exactly as they are.

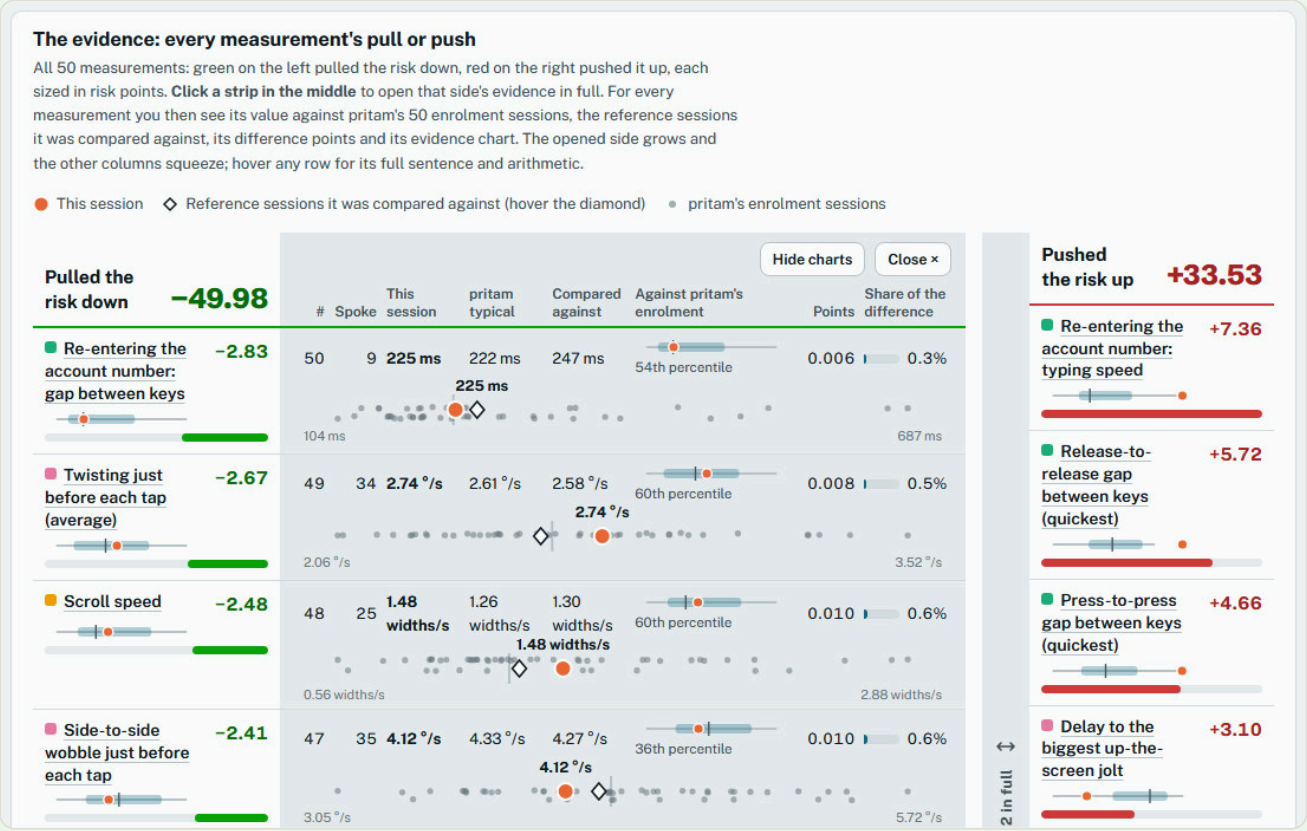
ML1. Keep the training rows inside each model file

ON THE DASHBOARD **Required**

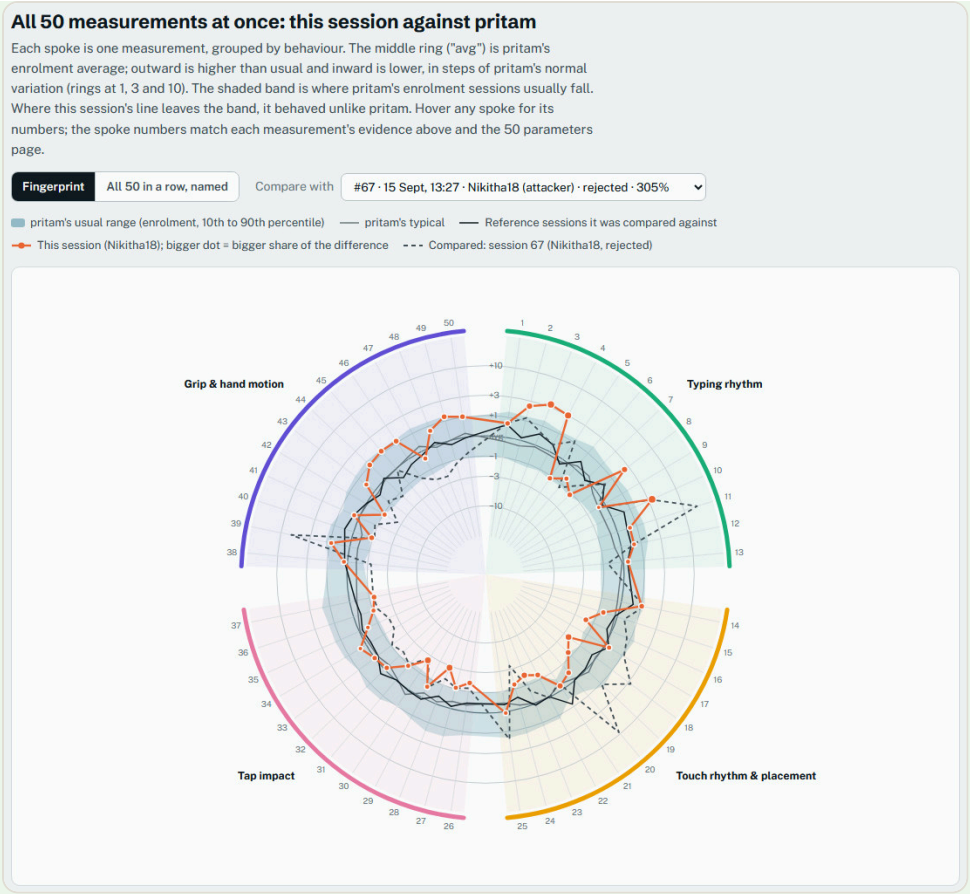
Everything that compares this session with the owner's enrolment.

- The **enrolment range bar** and **percentile** in every evidence row ("54th percentile").
- The **"typical"** column: the owner's median for each measurement.
- The **mini range** drawn inside every row of the pull/push lists.
- The **fingerprint**'s enrolment band and the "32 of the 50 measurements were inside the usual range" sentence in **Why**.

Without it: Users enrolled after the release would show their verdict and breakdown with every range, percentile and band empty.



Example from today's Verdict Explorer (research data): an opened evidence side. The "pritam typical" and "Against pritam's enrolment" columns come from the training rows.



Example from today's Verdict Explorer (research data): the fingerprint. Its grey enrolment band is drawn from the training rows.

File: `mobile_navigation_core/enrollment_service.py`, function `build_bundle()`, line 719.

Why: the page's enrolment ranges, "typical" values, percentiles, mini ranges and fingerprint all come from the rows the model was trained on. Today those rows live only in the training CSV in the staging folder, and `enrol()` deletes that folder at the end of every run (line 943). Only the model and the DTW templates survive.

Before:

```
    bundle_path = model.save(model_dir)
    raw = joblib.load(bundle_path)
```

After:

```
    bundle_path = model.save(model_dir)
    raw = joblib.load(bundle_path)
    # Verdict Explorer: the rows the SVM was fitted on (train_cols order) and the
    # session each row came from. model.support_ indexes into these rows.
    raw['train_values'] = np.asarray(X, dtype=np.float64)
    raw['train_session_ids'] = df['session_id'].astype(str).tolist()
```

Notes:

- `X` is exactly what `model.fit()` received (line 688): raw units, with NaN turned into 0 by `_matrix()`. Row i of `X` belongs to `train_session_ids[i]`.
- About 20 KB more per model (50 rows × 50 features).
- `publish()` never overwrites an existing model (line 849), so these rows never change after a model is built.
- `numpy` is already imported in this file (line 52).

ML2. Let the loaded model carry those rows

> **ON THE DASHBOARD** **Required, not visible** Enabler for ML1 and ML5
>
> **Nothing by itself. It makes the stored training rows and the calibration scores readable, so ML5 can send them.**
>
> - Feeds the **"Practice rejected 5%"** tile in the risk card (the calibration scores).
> - Feeds everything listed under ML1.
>
> **Without it:** ML1's rows would be saved but could not be read back.

File: `mobile_navigation_core/bundle_loader.py`, class `Bundle`, lines 72 and 93. `Bundle` uses `__slots__`, so every new attribute must be listed there.

Before:

```
    __slots__ = ('user_id', 'model', 'scaler', 'feature_columns', 'threshold',
                 'params', 'provenance', 'frozen_state', 'path', 'n_features')
    ...
        self.n_features = len(self.feature_columns)
```

After:

```
    __slots__ = ('user_id', 'model', 'scaler', 'feature_columns', 'threshold',
                 'params', 'provenance', 'frozen_state', 'path', 'n_features',
                 'train_values', 'train_session_ids', 'loo_scores')
    ...
        self.n_features = len(self.feature_columns)
        tv, ids = raw.get('train_values'), raw.get('train_session_ids')
        if tv is None:                    # built before ML1: sidecar file from ML6,
if any
            side = path[:-len(BUNDLE_SUFFIX)] + '_train_values.joblib'
            if os.path.exists(side):
                extra = joblib.load(side)
                tv, ids = extra['train_values'], extra['train_session_ids']
        self.train_values = None if tv is None else np.asarray(tv, dtype=np.float64)
        self.train_session_ids = ids
        self.loo_scores = raw.get('loo_scores')     # the calibration scores
```

`os`, `joblib` and `numpy` are already imported in this file. Scoring does not use these attributes.

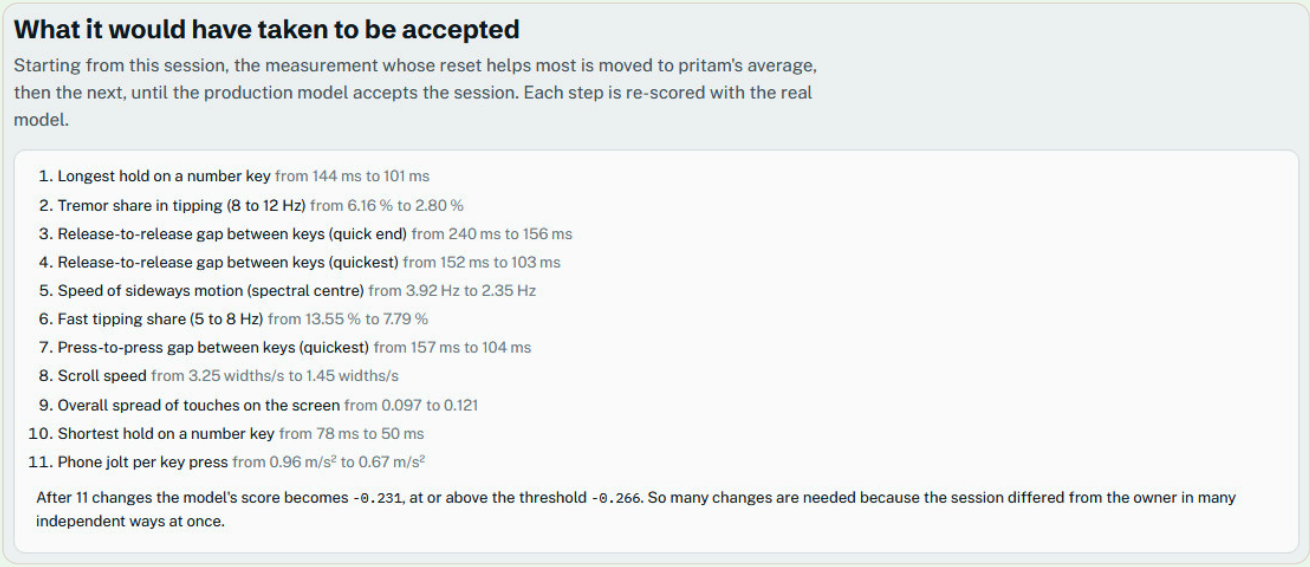
ML3. New module that computes the breakdown

ON THE DASHBOARD **Required** Needed in option A and option B

The maths behind every session explanation. Option A runs it inside predict (ML4); option B runs it in a separate explain endpoint.

- Each measurement's **penalty**, from which the pull/push points are made.
- The **"compared against"** values and the **reference weights**.
- **"What it would have taken to be accepted"** for rejected sessions.
- The self-checks that prove the breakdown matches the verdict.

Without it: No per-session explanation at all: only the verdict and the risk number.



Example from today's Verdict Explorer (research data): a rejected attack (pritam's session 51). This list is the counterfactual computed by ML3.

New file: `mobile_navigation_core/explain_verdict.py`.

Copy four functions unchanged from `explain_dashboard/build_explanations.py`, lines 244–325: `_logsumexp`, `explain`, `log_s_of` and `counterfactual`. These are the functions that produced every number in the demo. On 868 research sessions they reproduced production's score to within 4e-13. Only the wrapper below is new.

```
import math

import numpy as np

# _logsumexp, explain, log_s_of, counterfactual:
# copied unchanged from explain_dashboard/build_explanations.py (lines 244-325)

DIAG_KEYS = ('san_verdict', 'san_reason', 'san_repaired', 'san_streams_dead',
             'bc_verdict', 'n_touch', 'n_key', 'n_nav')
```

```
def _num(v):
    """JSON-safe float: inf and nan become None."""
    return float(v) if v is not None and math.isfinite(v) else None


def breakdown(bundle, row, diagnostics):
    """The exact explanation of one verdict, for the Verdict Explorer."""
    x = bundle.vectorise(row)[0]            # exactly the vector bundle.score() used
    ex = explain(bundle, x)
    w = ex['w']
    sv_raw = bundle.scaler.inverse_transform(bundle.model.support_vectors_)
    out = {
        'model_built_at': (bundle.provenance or {}).get('built_at'),
        'x': x.round(8).tolist(),           # this session's values
        'c': ex['c'].tolist(),              # each feature's penalty, sums to
deviation
        'w': w.tolist(),                    # weight of each reference session, sums
to 1
        'ref_raw': (w[:, None] * sv_raw).sum(axis=0).round(8).tolist(),
        'genuine_exact': bool(ex['genuine_exact']),
        'diag': {k: diagnostics.get(k) for k in DIAG_KEYS},
    }
    for k in ('deviation', 'allowance', 'need', 'rho', 'support', 'entropy',
              'log_s', 'ratio', 'score_recomputed', 'identity_error'):
        out[k] = _num(ex[k])
    if not ex['genuine_exact']:             # rejected: which features would flip it
        out['counterfactual'] = counterfactual(bundle, ex['z'], ex)
    return out
```

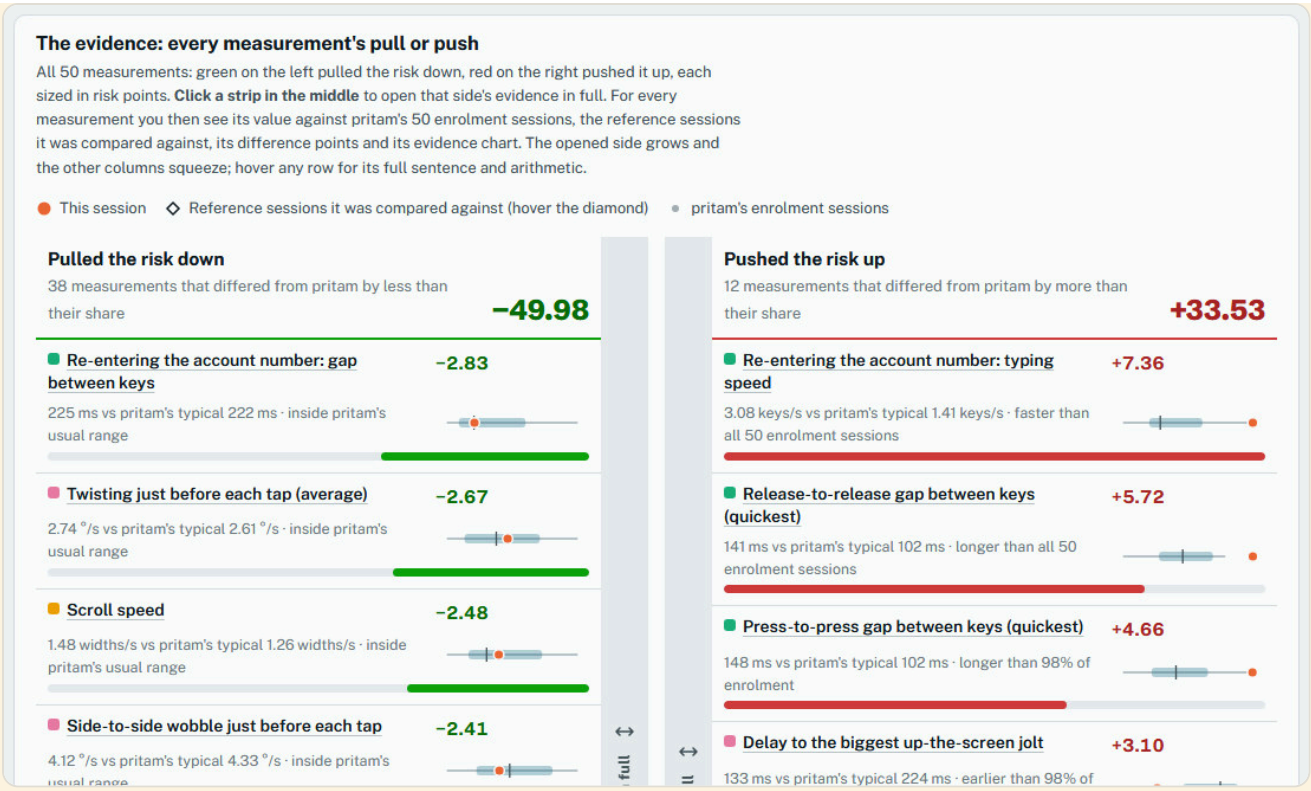
ML4. Return the breakdown from every predict

ON THE DASHBOARD **Optional** Optional as a set with DB1–DB3; required for the person page

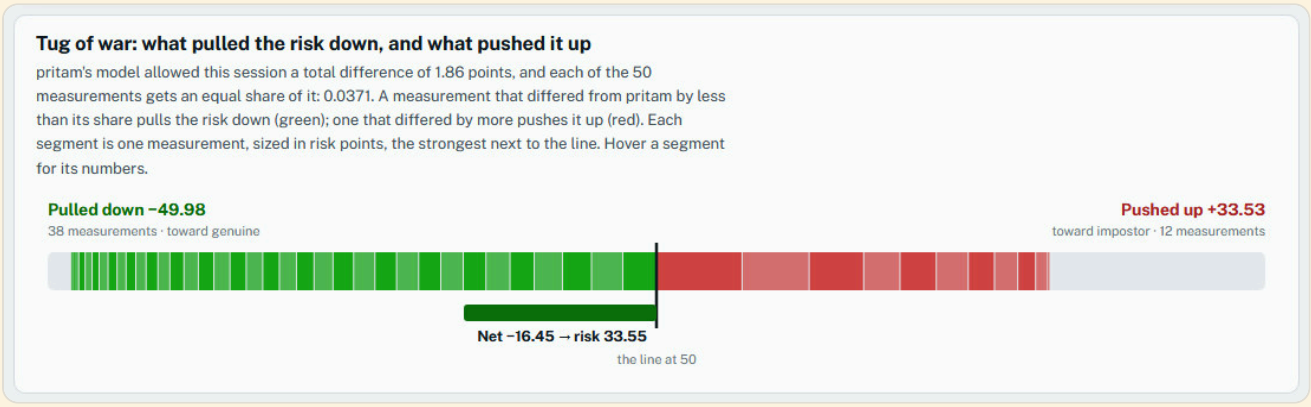
The session's breakdown, computed in the same request as the verdict and stored with it.

- **Pull/push lists with points**, tug of war, group waterfall, **Why** text, exact arithmetic, "compared against", reference weights, counterfactual. Option B can also show these, by re-extracting the session when the page opens.
- **Only ML4:** the person page's session list with deviation, allowance and ratio; the **"All sessions in time order"** chart; the dots on the S-curve; the behaviour-group summaries. These need a breakdown for every session at once.
- **Only ML4:** old sessions stay explainable after a later ML release changes the feature code.

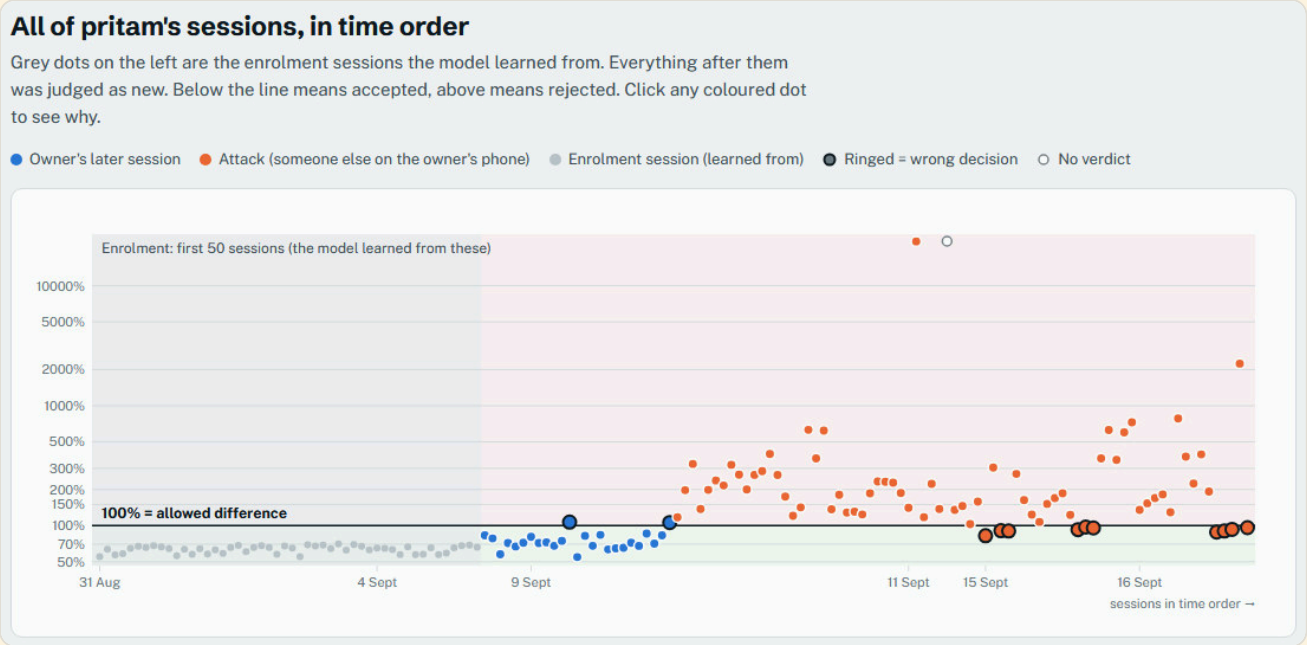
Without it: Choose option B (on-demand explain endpoint): single session pages still work, slower, with the person-page views missing.



Example from today's Verdict Explorer (research data): pritam's session 69. Every green and red number is a risk point made from the breakdown: 50 − 49.98 + 33.53 = 33.55.



Example from today's Verdict Explorer (research data): the tug of war: the same points as one picture.



Example from today's Verdict Explorer (research data): the person page. One dot per session, placed by its breakdown; needs breakdowns for all sessions, so only ML4 can feed it.

File: `mobile_navigation_core/prediction_service.py`, line 282, after the response is built and before the log line.

Before:

```
        if diagnostics.get('feat_n_errors'):
            response['feat_n_errors'] = diagnostics['feat_n_errors']

        logger.info(
```

After:

```
        if diagnostics.get('feat_n_errors'):
            response['feat_n_errors'] = diagnostics['feat_n_errors']
        if os.getenv('ML_EXPLAIN', 'true').strip().lower() == 'true':
            from explain_verdict import breakdown
            try:
                response['explain'] = breakdown(bundle, row, diagnostics)
            except Exception:                          # noqa: BLE001
                # Never let the explanation cost a verdict.
                logger.exception('explain failed for %s/%s', user_id, session_id)

        logger.info(
```

Rules:

- It runs after the verdict is decided, so it cannot change `is_genuine`, `score`, `threshold`, `margin` or `risk_score`.
- Any error is logged and the reply goes out without `explain`.

- `ML_EXPLAIN=false` in `.env` turns it off without a redeploy of code.
- Cost: one more pass over the support vectors (the same size as scoring), plus the counterfactual for rejected sessions only. The reply grows by about 4–5 KB. Measure both before switching it on (see the checks below).

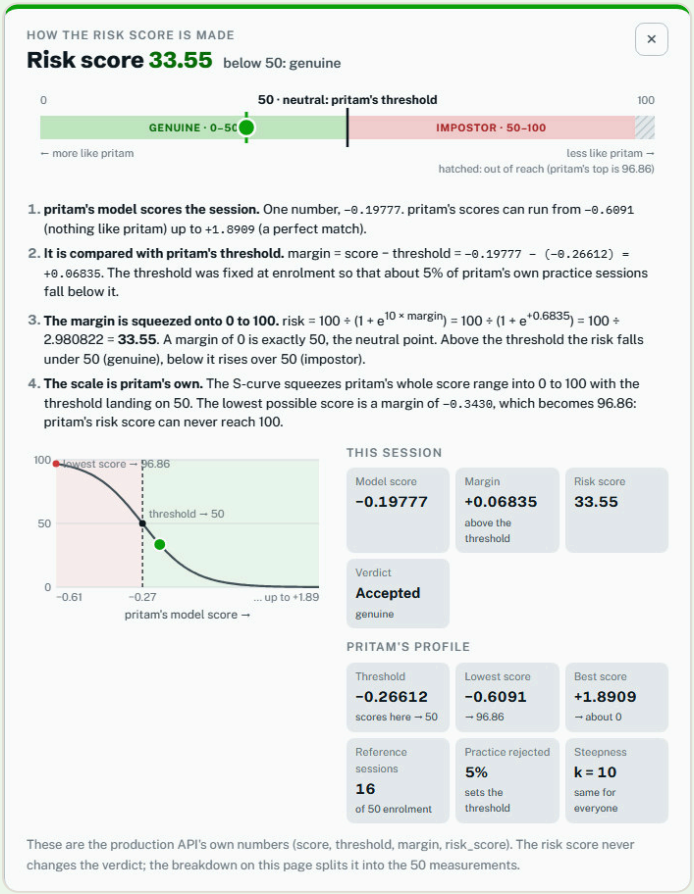
ML5. New endpoint: one user's model

ON THE DASHBOARD **Required** Model data exists only in the model file

Everything about the user's model rather than about one session.

- The **risk card**: the S-curve over the user's own score range, and the profile tiles (threshold, lowest and best score, reference sessions, practice rejected).
- The **highest possible risk** (pritam: 96.86), the hatched area on the gauge.
- The conversion from difference points to **risk points** (it needs ρ and α).
- The training rows for ML1's ranges, and the **reference sessions** list (which enrolment sessions the model kept).
- The model warning ("ceiling probe").

Without it: Only the verdict and the risk number. The dashboard backend cannot read this data elsewhere: Oracle holds only the threshold.



Example from today's Verdict Explorer (research data): the risk card. The lower tiles, the S-curve and steps 1 and 4 come from the model card.



Which of pritam's reference sessions it was compared against

The model keeps 16 of pritam's enrolment sessions as reference points. The difference is measured mostly against the references this session is closest to. The weight says how much each reference counted for this session.

Reference session	Recorded	Weight for this session
Enrolment session 5 of 50	31 Aug, 13:13	15.7%
Enrolment session 37 of 50	4 Sept, 13:12	14.5%
Enrolment session 33 of 50	2 Sept, 12:23	13.3%
Enrolment session 29 of 50	2 Sept, 12:11	9.4%
Enrolment session 26 of 50	2 Sept, 11:55	8.2%
Enrolment session 48 of 50	4 Sept, 13:29	7.7%

Example from today's Verdict Explorer (research data): the reference sessions. Which sessions they are comes from the model card; the weights come from the breakdown.

Add `GET /nav/ai/card/<user_id>`, protected by the same `X-ML-Train-Key` header as `/nav/ai/train`. A user's model never changes after it is published, so callers can cache this reply until `built_at` changes.

New folder `mobile_navigation_model_card/` with an empty `__init__.py`, and:

```
# mobile_navigation_model_card/mobile_navigation_card.py
"""GET /nav/ai/card/<user_id>: one user's model, in the form the Verdict Explorer
draws."""
import json

from flask import Response, jsonify
from flask.views import MethodView

from common import service_registry
from common.responses import service_unavailable, train_auth_error


class MobileNavigationCard(MethodView):

    def get(self, user_id):
        denied = train_auth_error()                 # X-ML-Train-Key, as /train
        if denied:
            return denied
        service = service_registry.prediction_service
        if service is None:
            return service_unavailable()
        from bundle_loader import ArtifactError
        from explain_verdict import explain, _num
        try:
            bundle, _ = service.store.get(user_id)
        except ArtifactError as error:
            return jsonify({'status': 'error', 'message': str(error)[:500]}), 404

        mdl, ids, tv = bundle.model, bundle.train_session_ids, bundle.train_values
```

```
        prov = bundle.provenance or {}
        enrol_scored = []
        if tv is not None:                          # how each training session itself
scores
            scores = mdl.decision_function(bundle.scaler.transform(tv))
            for i, sid in enumerate(ids):
                ex = explain(bundle, tv[i])
                enrol_scored.append({'session_id': sid, 'score': float(scores[i]),
                                     'deviation': _num(ex['deviation']),
                                     'allowance': _num(ex['allowance']),
                                     'ratio': _num(ex['ratio'])})
        card = {
            'status': 'success', 'user': user_id, 'built_at': prov.get('built_at'),
            'features': bundle.feature_columns, 'threshold': bundle.threshold,
            'rho': float(mdl.offset_[0]), 'gamma': float(mdl._gamma),
            'nu': bundle.params.get('nu'), 'params': bundle.params,
            'n_sv': int(len(mdl.support_)), 'alpha': mdl.dual_coef_[0].tolist(),
            'support_raw':
bundle.scaler.inverse_transform(mdl.support_vectors_).round(6).tolist(),
            'support_session_ids': [ids[i] for i in mdl.support_] if ids else None,
            'scaler_mean': bundle.scaler.mean_.tolist(),
            'scaler_scale': bundle.scaler.scale_.tolist(),
            'calibration_scores': bundle.loo_scores,
            'calibrate_method': bundle.params.get('calibrate_method'),
            'calibrate_frr': prov.get('calibrate_frr'),
            'provenance': prov,
            'train_ids': ids,
            'train_values': None if tv is None else tv.round(6).tolist(),
            'enrol_scored': enrol_scored,
        }
        # default=str: provenance holds a few values JSON cannot write directly
        return Response(json.dumps(card, default=str), mimetype='application/json')
```

```
# mobile_navigation_model_card/view.py
from flask import Blueprint

from mobile_navigation_model_card.mobile_navigation_card import MobileNavigationCard

mobile_navigation_model_card_api = Blueprint('mobile_navigation_model_card_api',
__name__)
mobile_navigation_model_card_api.add_url_rule(
    "/nav/ai/card/<user_id>",
view_func=MobileNavigationCard.as_view('mobile_navigation_model_card'),
    methods=['GET'])
```

In `app.py`, next to the other four blueprints:

```
from mobile_navigation_model_card.view import mobile_navigation_model_card_api
...
app.register_blueprint(mobile_navigation_model_card_api)
```

nginx already forwards every path to gunicorn, so it needs no change.

ML6. Fill in the training rows of models built before ML1

> ON THE DASHBOARD **Optional** Only for users enrolled before ML1
>
> **ML1's parts for users who already have a model on release day.**
>
> - Enrolment ranges, percentiles, mini ranges and the fingerprint band for those existing users.
>
> **Without it:** Existing users show verdicts and breakdowns, but no enrolment ranges. Users enrolled later are not affected.

A one-off script, `rebuild_train_values.py`, for every model file that has no `train_values`:

1. Fetch the user's sessions exactly as `enrol()` does (line 868), using the `profile_id` recorded in the model's provenance.
2. Run `build_bundle()` into a temporary staging folder. This trains a copy but never publishes it.
3. Accept the copy only if its `scaler.mean_`, `scaler.scale_` and `support_vectors_` equal the live model's (to 1e-12). Equal values prove it trained on the same rows.
4. Write `<user>_train_values.joblib` (`train_values`, `train_session_ids`) next to the live model. ML2 reads it. The live model file is never touched.

If the values do not match, the user's stored sessions changed since enrolment. Report that user: the page then shows their verdicts and breakdowns, but no enrolment ranges.

ML7. Keep the v8 folders on persistent storage

> ON THE DASHBOARD **Required, not visible** Protects every page
>
> **Nothing new. It keeps the v8 models when the container is recreated.**
>
> - Every user's page keeps working after a restart or redeploy.
>
> **Without it:** After a container is recreated, every v8 user loses their model: no verdicts, no model card, nothing to show.

With `ML_SELECTION=v8`, models go to `/app/saved_models_v8`, templates to `/app/templates_v8` and jobs to `/app/jobs_v8` (`selections.py`, line 170). `docker-compose.yml` only persists `/app/saved_models` and `/app/templates`, so v8 models are lost whenever the container is recreated. Check the ECS task definition the same way.

Before:

```
    volumes:
      - trained_models:/app/saved_models
      - trained_templates:/app/templates
```

After:

```
    volumes:
      - trained_models:/app/saved_models           # kept: the rev4 profile still
uses it
      - trained_templates:/app/templates
      - trained_models_v8:/app/saved_models_v8     # new: where v8 models are written
      - trained_templates_v8:/app/templates_v8
      - train_jobs_v8:/app/jobs_v8

volumes:
  trained_models:
  trained_templates:
  trained_models_v8:
  trained_templates_v8:
  train_jobs_v8:
```

ML checks before handing over

- On the 43 captured uploads and the 868 research sessions, `is_genuine`, `score`, `threshold`, `margin` and `risk_score` are identical with `ML_EXPLAIN` on and off.
- `explain.score_recomputed` equals `score` within 1e-9, and `identity_error` is below 1e-9.
- Report the median and 95th-percentile predict time with `ML_EXPLAIN` on and off, and the reply size.
- `/nav/ai/card/<user_id>` returns 401 without the key and 404 for an unknown user.

What the ML team hands over: the `explain` block

Every successful predict reply gains one field, `explain`. Lists follow the model's `feature_columns` order; `w` follows the support-vector order of the model card.

Field	Type	Meaning	Page uses it for
`model_built_at`	text	`built_at` of the model that scored	Matching the breakdown to the model card

Field	Type	Meaning	Page uses it for
`x`	50 numbers	The session's raw feature values	"This session" column, mini ranges
`c`	50 numbers	Each feature's penalty (0 or more); their sum is `deviation`	Pull/push points, tug of war, waterfall, why text
`w`	one number per reference session	How much each reference session supports this one; sums to 1	Reference sessions section
`ref_raw`	50 numbers	The weighted reference values	"Compared against" column
`deviation`, `allowance`, `need`	numbers	D, A and threshold + ρ in the exact identity	The risk breakdown and its arithmetic
`rho`, `support`, `entropy`, `log_s`, `ratio`	numbers	The rest of the identity	Exact arithmetic section
`score_recomputed`, `identity_error`, `genuine_exact`	number, number, true/false	Self-checks	Proof the breakdown matches the verdict
`diag`	object	Sanitizer verdict and reason, event counts	Data-quality notes
`counterfactual`	object, rejected sessions only	`order` (feature indexes), `flipped`, `final_score`	"What would flip the verdict"

The model card from ML5 carries everything per user: threshold, ρ, γ, ν, α, support vectors and their session IDs, scaler, calibration scores, provenance, training rows and IDs, and `enrol_scored`.

DB backend team

The DB backend stores the ML's `explain` block next to each verdict, in its own column, and fills it in for sessions scored before the change. Repository: the backend (`backend (5)/backend`). The mobile app's response does not change: only `is_genuine`, `swipe_prediction` and `risk_score` are passed back to it (`mobile_pattern_core_svm_api.py`, line 106; `utils/authn_store.py`, line 161).

Deploy DB2 before ML4 is switched on. Today `save_session_score()` writes the whole ML reply to `DETAILS_JSON` and cuts it to four fields when it is longer than 4,000 characters. With the 4–5 KB `explain` block inside, every reply would be cut. The ML team can deploy ML4 with `ML_EXPLAIN=false` and switch it on once DB2 is live.

DB1. New column for the breakdown

ON THE DASHBOARD **Optional** Optional as a set with ML4

The place where ML4's breakdown is kept.

- Everything ML4 shows, read from Oracle in one query instead of being recomputed.

Without it: Fine if you choose option B.

```
ALTER TABLE ADAPID.NAV_SESSION_SCORES ADD (EXPLAIN_JSON CLOB);

-- rollback
ALTER TABLE ADAPID.NAV_SESSION_SCORES DROP COLUMN EXPLAIN_JSON;
```

The column is nullable, so existing rows and the current code keep working before DB2 is deployed. Expect about 5 KB per scored session.

DB2. Save the breakdown with every verdict

ON THE DASHBOARD **Optional** Must ship with ML4

Saves ML4's breakdown with each verdict, and keeps the existing `DETAILS_JSON` intact.

- Everything ML4 shows, for every session scored after the release.

Without it: If ML4 is switched on without DB2, every session's `DETAILS_JSON` is cut to four fields. Skip both, or do both.

File: `utils/pattern_api_services.py`, function `save_session_score()`, line 172. Both scoring paths save through this one function: the direct path (`mobile_pattern_core_svm_api.py`, line 102) and the Celery path (`utils/authn_store.py`, line 74).

Before (the parts that change):

```
    details = None
    ai = result.get('ai')
    if ai is not None:
        details = json.dumps(ai)
        if len(details) > 4000:
            details = json.dumps({**{k: ai.get(k) for k in ('risk_score', 'score',
'threshold', 'margin')}, 'truncated': True})
```

```
    ...
        cursor = db_conn.cursor()
        cursor.execute(
            f"UPDATE {db_name}.NAV_SESSION_SCORES SET ML_STATUS = :status, IS_GENUINE
= :genuine, SWIPE_PREDICTION = :swipe, "
            f"RISK_SCORE = :risk, DETAILS_JSON = :details, ERROR = :err, DURATION_MS =
:dur, QUEUED_AT = :queued, "
            f"SCORED_AT = :scored, UPDATED_AT = CURRENT_TIMESTAMP WHERE SESSION_ID =
:sid", binds)
        if cursor.rowcount == 0:
            cursor.execute(
                f"INSERT INTO {db_name}.NAV_SESSION_SCORES (SESSION_ID, USER_ID,
ML_STATUS, IS_GENUINE, SWIPE_PREDICTION, "
                f"RISK_SCORE, DETAILS_JSON, ERROR, DURATION_MS, QUEUED_AT, SCORED_AT)
"
                f"VALUES (:sid, :user_id_bind, :status, :genuine, :swipe, :risk,
:details, :err, :dur, :queued, :scored)",
                {**binds, 'user_id_bind': user_id})
```

After:

```
    details = explain = None
    ai = result.get('ai')
    if ai is not None:
        ai = dict(ai)                                   # a copy: the caller's verdict
is not changed
        block = ai.pop('explain', None)                 # too big for DETAILS_JSON:
its own column
        explain = None if block is None else json.dumps(block)
        details = json.dumps(ai)
        if len(details) > 4000:
            details = json.dumps({**{k: ai.get(k) for k in ('risk_score', 'score',
'threshold', 'margin')}, 'truncated': True})
    ...
    binds = {
        ...                                             # every existing key,
unchanged
        'explain': explain,
    }
    ...
        cursor = db_conn.cursor()
        cursor.setinputsizes(explain=oracledb.DB_TYPE_CLOB)
        cursor.execute(
            f"UPDATE {db_name}.NAV_SESSION_SCORES SET ML_STATUS = :status, IS_GENUINE
= :genuine, SWIPE_PREDICTION = :swipe, "
            f"RISK_SCORE = :risk, DETAILS_JSON = :details, EXPLAIN_JSON = :explain,
ERROR = :err, DURATION_MS = :dur, "
            f"QUEUED_AT = :queued, SCORED_AT = :scored, UPDATED_AT = CURRENT_TIMESTAMP
WHERE SESSION_ID = :sid", binds)
```

```
        if cursor.rowcount == 0:
            cursor.setinputsizes(explain=oracledb.DB_TYPE_CLOB)
            cursor.execute(
                f"INSERT INTO {db_name}.NAV_SESSION_SCORES (SESSION_ID, USER_ID,
ML_STATUS, IS_GENUINE, SWIPE_PREDICTION, "
                f"RISK_SCORE, DETAILS_JSON, EXPLAIN_JSON, ERROR, DURATION_MS,
QUEUED_AT, SCORED_AT) "
                f"VALUES (:sid, :user_id_bind, :status, :genuine, :swipe, :risk,
:details, :explain, :err, :dur, :queued, :scored)",
                {**binds, 'user_id_bind': user_id})
```

Add `import oracledb` at the top of the file (it is already in `requirements.txt`).

What changes:

- `DETAILS_JSON` holds exactly what it holds today, because the block is taken out before the 4,000-character check.
- A failed or missing breakdown leaves `EXPLAIN_JSON` empty; the verdict is still saved.
- On the Celery path the verdict, block included, travels in the task message, so each message grows by about 5 KB.

DB3. Fill in sessions scored before the change

> **ON THE DASHBOARD** **Optional** Only for older sessions
>
> **Breakdowns for sessions scored before the release.**
>
> - ML4's parts on pages of sessions from before release day.
>
> **Without it:** Those sessions show their verdict and risk number with the note "The breakdown is not available for this session."

A one-off script, run once after ML4 and DB2 are live. For each successful score with no breakdown, it rebuilds the stored session, asks the ML for its verdict again, and saves the block only if the new score equals the stored one. Equal scores prove the same model scored the same features. Predict writes nothing, so asking again is safe.

```
# backfill_explain.py: run from the backend repo root, off-peak
import json
import os

from mysql_utils.db_connection import get_oracle_connection
from utils.nav_session_helpers import SESSION_SELECT, assemble_sessions, fetch_dicts
from utils.pattern_api_services import call_predict_api

db_name = os.getenv('ORACLE_DB_NAME')
conn = get_oracle_connection()
todo = fetch_dicts(conn, f"SELECT SESSION_ID, DETAILS_JSON FROM
{db_name}.NAV_SESSION_SCORES "
```

```
                          f"WHERE ML_STATUS = 'success' AND EXPLAIN_JSON IS NULL")
done = mismatch = failed = 0
for n, row in enumerate(todo, 1):
    stored = json.loads(row['DETAILS_JSON'] or '{}')
    rows = fetch_dicts(conn, SESSION_SELECT.format(db=db_name) + "WHERE s.SESSION_ID =
:sid",
                       {'sid': row['SESSION_ID']})
    # the same shape payload_from_request_session() sends (see its docstring)
    payload = assemble_sessions(conn, db_name, rows, include_events=True,
with_profile=True)[0]
    result = call_predict_api(payload)
    ai = result.get('ai') or {}
    if result['ml_status'] != 'success' or 'explain' not in ai:
        failed += 1
    elif ai.get('score') != stored.get('score'):
        mismatch += 1                                  # different model or different
features: leave empty
    else:
        cur = conn.cursor()
        cur.setinputsizes(e=oracledb.DB_TYPE_CLOB)
        cur.execute(f"UPDATE {db_name}.NAV_SESSION_SCORES SET EXPLAIN_JSON = :e "
                    f"WHERE SESSION_ID = :sid AND EXPLAIN_JSON IS NULL",
                    {'e': json.dumps(ai['explain']), 'sid': row['SESSION_ID']})
        conn.commit()
        done += 1
    if n % 50 == 0 or n == len(todo):
        print(f'{n}/{len(todo)}  saved={done}  score_mismatch={mismatch}  failed=
{failed}', flush=True)
```

(Also `import oracledb`.) Report the three counts when it finishes. A user's model is never overwritten once published, so mismatches should only come from users whose model file was deleted and rebuilt by hand, or from sessions whose stored copy differs from what the app sent.

DB checks before handing over

- On staging, `DETAILS_JSON` for new sessions is byte-for-byte what it was before DB2.
- Every new successful score has `EXPLAIN_JSON`, and it parses as JSON.
- The mobile app's response body is unchanged.
- The backfill's three counts are reported.

What the DB team hands over

`NAV_SESSION_SCORES.EXPLAIN_JSON`: the ML's `explain` block, exactly as the ML returned it (the table in the ML section lists its fields). It sits beside the existing `RISK_SCORE`, `IS_GENUINE`, `DETAILS_JSON` (score, threshold, margin and the rest of the reply) and `COMBINE_RISK_SCORE`.

Dashboard backend team

The dashboard backend serves three login-protected APIs that return the page's data already in its final shape: one user, one session, and a user's session list. Today `build_dashboard.py` and `build_dashboard_risk.py` do this shaping offline. The same Python moves into the backend, so the page's maths does not change. The code lives in the backend repository, next to the existing dashboard APIs in `dashboard_behaviour_api/`.

DA1. Fetch a user's model from the ML, with a cache

ON THE DASHBOARD **Required**

Brings ML5's model data to the page, cached per user.

- Everything listed under ML5: the risk card profile, the S-curve, the ceiling, the ranges, the reference sessions.

Without it: ML5 would exist but nothing would reach the page.

File: `utils/pattern_api_services.py`, next to `call_train_api()`. It uses the existing `_ai_nav_url()` and `_train_headers()` (the `X-ML-Train-Key` header).

```
_CARD_CACHE = {}                        # user_id -> (fetched_at, card)
_CARD_TTL_S = 3600                      # a published model never changes; re-check
hourly


def get_model_card(user_id):
    """GET {AI_NAV_URL}/card/<user_id> (ML5). None when the ML has no model for the
user."""
    hit = _CARD_CACHE.get(user_id)
    if hit and time.time() - hit[0] < _CARD_TTL_S:
        return hit[1]
    url = f"{_ai_nav_url()}/card/{requests.utils.quote(user_id, safe='')}"
    response = requests.get(url, headers=_train_headers(),
timeout=float(os.getenv('ML_TIMEOUT_S', 45)))
    if response.status_code != 200:
        return None
    card = response.json()
    _CARD_CACHE[user_id] = (time.time(), card)
    return card
```

Each gunicorn worker keeps its own cache, which is fine: a card is about 50 KB and changes only if a model is rebuilt by hand.

DA2. Move the page's shaping code into the backend

ON THE DASHBOARD **Required**

Turns stored rows into exactly the numbers the page prints.

- Risk points per measurement that **add up exactly** to the risk score, including the "shown as" rounding notes.
- Group totals for the waterfall; the inputs to the **Why** text.
- The **exact arithmetic** block. A breakdown that fails any check is never shown.

Without it: The page would have raw numbers it cannot draw.



Example from today's Verdict Explorer (research data): the group waterfall and the S-curve. Group totals are summed by DA2; the curve's dots come from the session list.

− The exact arithmetic of this risk score

```
risk_score = 100 · (1 − 1 / (1 + e^(−10 · (score − threshold))))          (production code)
           = 100 · (1 − 1 / (1 + e^(−10 × 0.06835119)))  = 33.547793  → returned as 33.55

score − threshold = need · (e^(A − D) − 1)                                (an identity, no approximation)
                  = 0.343021 × (e^(1.85550 − 1.67380) − 1)  = 0.06834837

D = Σⱼ cⱼ = 1.67380      the session's difference, one term per measurement
A         = 1.85550      its allowance = 1.98626 (pritam's maximum) − 0.13075 (reference mix)

Equal share of the allowance:  A / 50 = 0.037110
netⱼ    = cⱼ − A/50                    Σⱼ netⱼ = D − A = −0.181700
rate    = (33.5478 − 50) / (D − A) = 90.5426 risk points per difference point
pointsⱼ = netⱼ × rate                  Σⱼ pointsⱼ = −16.4522

50 + Σⱼ pointsⱼ = 33.5478  → 33.55, the API's risk_score ✓

Shown on this page, each part rounded to 0.01 so that they still add up exactly:
50 + 33.53 (12 pushes) − 49.98 (38 pulls) = 33.55
```

c_j is measurement j's exact difference points (shown in each measurement's evidence above). The equal share is the one convention: it sets each measurement's zero point, so it decides whether a small contributor counts as a pull or a push, but it changes no total and no ranking. The rate is a single number for the whole session because the risk score depends only on D − A. The reference mix is the part of the allowance that belongs to no single measurement: how evenly this session resembled pritam's reference sessions. The equal shares spread it over all 50 measurements.

Example from today's Verdict Explorer (research data): the exact arithmetic, which DA2's checks guarantee.

New files in `dashboard_behaviour_api/`:

- `glossary.py`: a copy of `explain_dashboard/glossary.py` (feature names, definitions, groups).
- `explain_shape.py`: the shaping code. Everything except two small functions is copied unchanged.

Copy unchanged	From
`sig`, `sigs`, `compact_replay`	`build_dashboard.py`, lines 36–68
`truth_of`, `_screens`, `session_meta`, `replay_of`	`build_explanations.py`, lines 93–146 and 327–361
`K = 10`, `production_risk`, `hundredths`, `breakdown`, `check_session`	`build_dashboard_risk.py`, lines 68–142
The `fstats` loop and the `model` block	`build_dashboard.py`, `build_user()`, lines 118–127 and 223–236
The `risk` block	`build_dashboard_risk.py`, `build_user()`, lines 162–166

The two new functions take production rows instead of research files. `shape_session` returns one element of the page's `tests` list, risk breakdown included:

```
def shape_session(card, sess, row, enrol_devices, order, fields):
    """One scored session, exactly as one element of build_user()['tests'].

    card: the ML model card (DA1)      sess: the session from assemble_sessions(..., include_events=True)
    row:  its NAV_SESSION_SCORES row    order: its position among the user's sessions by start time
    """
    details = json.loads(row['DETAILS_JSON'] or '{}')     # score, threshold, margin, risk_score, san_verdict ...
    ex = json.loads(row['EXPLAIN_JSON']) if row['EXPLAIN_JSON'] else None
    m = session_meta(sess)
    rep = replay_of(sess)
    fields = fields + sorted({k[2] for k in rep['keys']} - set(fields))   # new key fields go last
    ok = row['ML_STATUS'] == 'success'
    diag = (ex or {}).get('diag') or {}
    s = {
        'id': m['session_id'], 'o': order, 't': m['start_ts'], 'dur': m['duration_s'],
        'op': m['operator_id'], 'truth': _truth(sess),
        'nk': m['n_key'], 'nt': m['n_touch'], 'scr': m['screens'],
        'dev': 'same' if m['device_id'] in enrol_devices else 'other',
        'http': 200 if ok else None,
        'v': (row['IS_GENUINE'] == 'true') if ok else None,
```

```
        'san': details.get('san_verdict'), 'sanr': diag.get('san_reason') or '',
        'err': None if ok else (row['ERROR'] or '')[:300],
        'rp': compact_replay(rep, fields), 'fields': fields,
    }
    if not (ok and ex):
        return s                                            # no verdict, or scored
before ML4 and not backfilled
    w = np.asarray(ex['w'], dtype=float)
    s.update({
        'score': details['score'], 'thr': details['threshold'], 'margin':
details['margin'],
        'risk': details.get('risk_score'),
        'D': sig(ex['deviation'], 6), 'A': sig(ex['allowance'], 6), 'ratio':
sig(ex['ratio'], 6),
        'logS': sig(ex['log_s'], 6), 'sup': sig(ex['support'], 6), 'H':
sig(ex['entropy'], 6),
        'need': sig(ex['need'], 6),
        'x': sigs(ex['x'], 6), 'c': sigs(ex['c'], 4), 'ref': sigs(ex['ref_raw'], 6),
'w': sigs(ex['w'], 4),
        'refs': [[int(i), sig(w[i], 4)] for i in np.argsort(-w)[:3]],   # index into
the support list
        'chk': {'score_matches_response': abs(round(ex['score_recomputed'], 8) -
details['score']) < 1e-9,
                'verdict_matches_exact': ex['genuine_exact'] == (row['IS_GENUINE'] ==
'true'),
                'recompute_err': abs(ex['score_recomputed'] - details['score'])},
    })
    cf = ex.get('counterfactual')
    if cf:
        s['cf'] = {'order': cf['order'], 'flip': cf['flipped'], 'final':
sig(cf['final_score'], 6)}
    rec = dict(ex, response=dict(details, is_genuine=s['v']))
    b = breakdown(rec, card)
    problems = check_session(card['user'], s['id'], rec, b)
    if problems:                                           # never show a breakdown that
fails a check
        logger.error('risk breakdown failed: %s', problems[:3])
        s['rkError'] = True
    else:
        s['rk'] = {'full': sig(b['full'], 12), 'rate': sig(b['rate'], 8), 'share':
sig(b['share'], 8),
                   'mix': sig(b['mix'], 6), 'pts': sigs(b['pts'], 8), 'h': [int(v) for
v in b['h']]}
    return s


def _truth(sess):
    """'g' / 'i' only where the app really labels sessions (a test build); None in
production."""
    if os.getenv('EXPLAIN_TRUST_LABELS', 'false').lower() != 'true':
```

```
        return None
    return 'g' if truth_of(sess) == 'genuine' else 'i'
```

`shape_user(card, train_sessions, order_of, n_judged)` returns the page's user object. It is `build_user()` without `tests` and `summary`, built the same way:

- `features`, `fgroup` and `model` from the card; `fstats` from `card['train_values']`; `risk` from the card's threshold, ρ and α, with `n` = `n_judged`.
- `enrol`: one entry per `card['train_ids']`, with meta from `session_meta()`, scores from `card['enrol_scored']`, `sv` from `card['support_session_ids']`, and a replay for the support sessions only. `san` and `par` are `None`, because enrolment sessions are not re-checked by the sanitizer in production.
- `fields`: the key-field names seen in the training sessions, most common first (`field_codes()` logic).
- `devices`: session counts per device (counts only, no device IDs), and `enrolDevices`.

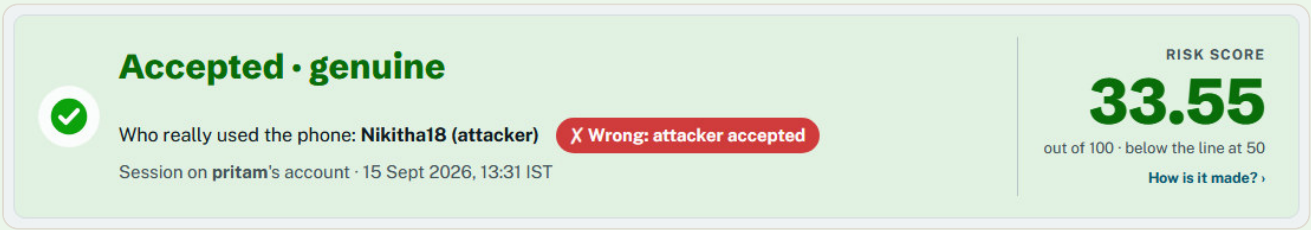
DA3. Three dashboard APIs

ON THE DASHBOARD **Required**

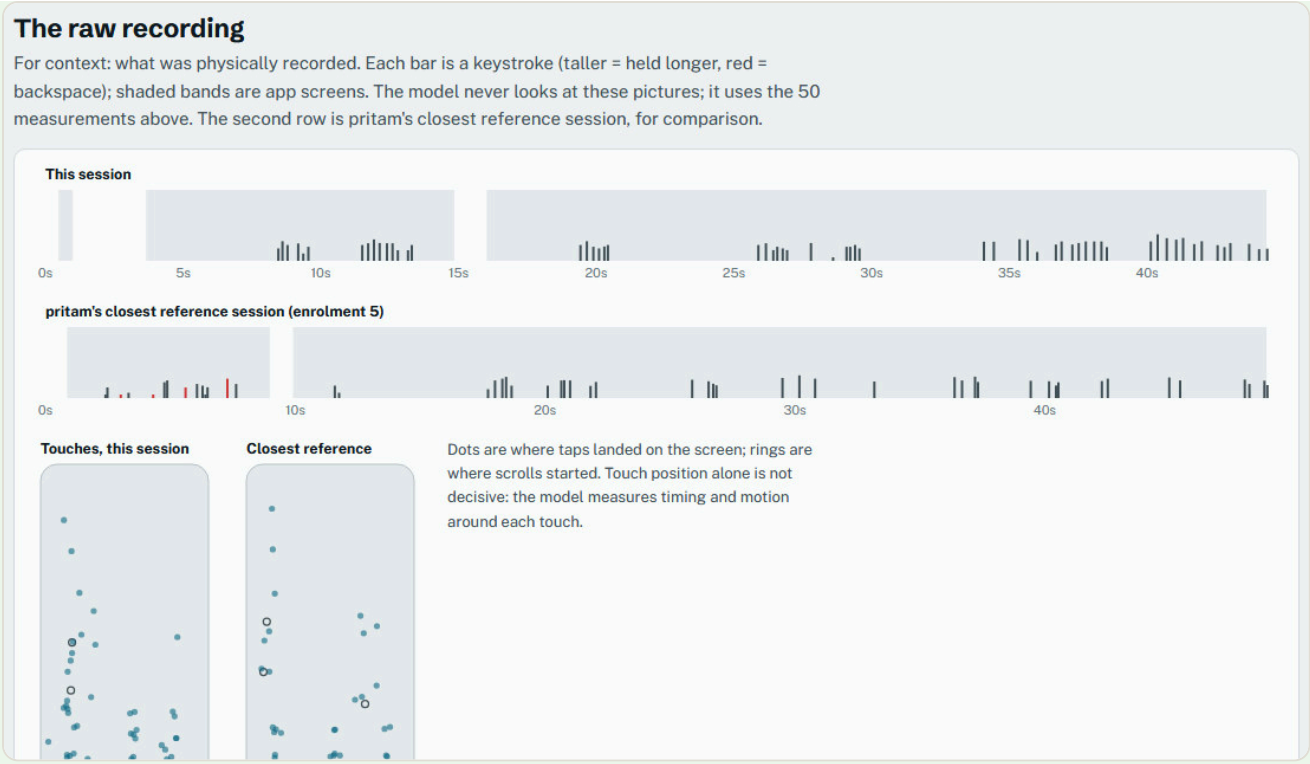
The three APIs the page loads from.

- **Session API:** the whole session page: banner, risk section, Why, fingerprint, counterfactual, references, replay.
- **User API:** the enrolment context: ranges, reference sessions, the risk card's profile.
- **Session list API:** the person page's table and charts.

Without it: No page can load.



Example from today's Verdict Explorer (research data): the session banner (verdict and risk score). In production the "Who really used the phone" line and the red badge are removed (FE4).



Example from today's Verdict Explorer (research data): the raw recording, rebuilt from the touch, key and navigation events already stored in Oracle.

New file `dashboard_behaviour_api/behaviour_explain_api.py`, written like `behaviour_logs_overview_api.py`: `@jwt_required()`, `ORACLE_DB_NAME`, `mysql_utils.db_connection.get_oracle_connection`, the same error bodies, and the connection closed in `finally`.

`GET /api/v4/explain/session?session_id=` returns `{"user": <user id>, "session": <shape_session(...)>}`.

```
class DashboardExplainSessionApi(MethodView):

    @jwt_required()
    def get(self):
        db_conn = None
        try:
            db_name = os.getenv('ORACLE_DB_NAME')
            session_id = (request.args.get('session_id') or '').strip()
            if not session_id:
                return {'message': 'Incorrect payload'}, 400
            db_conn = get_oracle_connection()
            scores = fetch_dicts(db_conn, f"SELECT USER_ID, ML_STATUS, IS_GENUINE,
RISK_SCORE, DETAILS_JSON, "
                                         f"EXPLAIN_JSON, ERROR FROM
{db_name}.NAV_SESSION_SCORES "
                                         f"WHERE SESSION_ID = :sid", {'sid':
session_id})
            if not scores:
                return jsonify({'status': 'error', 'message': 'Session not found'}),
404
            row = scores[0]
```

```
            card = get_model_card(row['USER_ID'])
            if card is None:
                return jsonify({'status': 'error', 'message': 'No model for this
user'}), 404
            rows = fetch_dicts(db_conn, SESSION_SELECT.format(db=db_name) + "WHERE
s.SESSION_ID = :sid",
                               {'sid': session_id})
            sess = assemble_sessions(db_conn, db_name, rows, include_events=True)[0]
            order = session_order(db_conn, db_name, row['USER_ID'],
sess['session_start_ts'])
            enrol_devices = training_device_ids(db_conn, db_name, card['train_ids'] or
[])
            fields = training_key_fields(row['USER_ID'])           # cached with the
user object
            return jsonify({'user': row['USER_ID'],
                            'session': shape_session(card, sess, row, enrol_devices,
order, fields)}), 200
        except Exception as e:
            logger.error(f'Error in DashboardExplainSessionApi: {e}')
            return jsonify({'status': 'error', 'message': 'Internal server error'}),
500
        finally:
            if db_conn:
                db_conn.close()
```

`session_order` is one `COUNT(*)` of the user's sessions that started earlier. `training_device_ids` is one `SELECT DISTINCT DEVICE_ID` over the training session IDs. `fetch_dicts`, `SESSION_SELECT` and `assemble_sessions` already exist in `utils/nav_session_helpers.py`; `fetch_dicts` already returns CLOBs as text.

`GET /api/v4/explain/user?user_id=` returns `shape_user(...)`. It loads the training sessions with `SESSION_SELECT` + `assemble_sessions(..., include_events=True)` and counts the user's successful scores. Cache the result per user next to the card: it is the heaviest call (about 50 sessions with their events) and changes only when the user gets new scored sessions (`risk.n`).

`GET /api/v4/explain/sessions?user_id=&start_date=&end_date=` (dates `DD-MM-YYYY`, both or neither, like the other dashboard APIs; default the last 30 days) returns light rows for the user page's table and its scatter charts. The numbers come out of the JSON columns inside Oracle, so no large text is sent to Python:

```
SELECT s.SESSION_ID, s.SESSION_START_TS, s.SESSION_END_TS, s.DEVICE_ID, s.OPERATOR_ID,
       sc.ML_STATUS, sc.IS_GENUINE, sc.RISK_SCORE,
       JSON_VALUE(sc.DETAILS_JSON, '$.score'     RETURNING NUMBER) AS SCORE,
       JSON_VALUE(sc.DETAILS_JSON, '$.margin'    RETURNING NUMBER) AS MARGIN,
       JSON_VALUE(sc.EXPLAIN_JSON, '$.deviation' RETURNING NUMBER) AS D,
       JSON_VALUE(sc.EXPLAIN_JSON, '$.allowance' RETURNING NUMBER) AS A,
       JSON_VALUE(sc.EXPLAIN_JSON, '$.ratio'     RETURNING NUMBER) AS RATIO
FROM {db}.NAV_SESSION_SCORES sc
JOIN {db}.NAV_SESSIONS s ON s.SESSION_ID = sc.SESSION_ID
```

```
WHERE sc.USER_ID = :u AND s.SESSION_START_TS BETWEEN :from_ms AND :to_ms
ORDER BY s.SESSION_START_TS
```

Each row maps to the page's keys `id`, `t`, `dur`, `op`, `dev`, `v`, `risk`, `score`, `margin`, `D`, `A`, `ratio`, with `truth` from `_truth()`. With `&summary=1` it also returns accepted, rejected and no-verdict counts, plus behaviour-group shares and top features over the period; computing those reads `EXPLAIN_JSON` in full, so cap the range at 500 sessions.

Routes, in `dashboard_behaviour_api/view.py` next to the `logs/overview` line (import the three classes at the top):

```
dashboard_apis_blueprint.add_url_rule("/api/v4/explain/session",
view_func=DashboardExplainSessionApi.as_view('dashboard_explain_session_api'),
methods=['GET'])
dashboard_apis_blueprint.add_url_rule("/api/v4/explain/user",
view_func=DashboardExplainUserApi.as_view('dashboard_explain_user_api'), methods=
['GET'])
dashboard_apis_blueprint.add_url_rule("/api/v4/explain/sessions",
view_func=DashboardExplainSessionsApi.as_view('dashboard_explain_sessions_api'),
methods=['GET'])
```

In production the paths take the `/mb` prefix (`/mb/api/v4/explain/...`), like the other dashboard APIs.

DA4. Access and privacy

> ON THE DASHBOARD **Required, not visible** Who may open the pages
>
> **Nothing visible. Only dashboard users with the right role can load the pages.**

- All three APIs need the dashboard login (`@jwt_required()`). Limit them to the roles that may already see behaviour logs, through the existing role permissions.
- The responses hold raw behaviour measurements and session replays. Never log response bodies, and keep the ML card call server-side: the browser never sees the `X-ML-Train-Key`.

DA5. Labels: keep `truth` empty in production

> ON THE DASHBOARD **Changes the page** Labels stay empty in production
>
> **The page no longer claims who really used the phone.**
>
> - The owner/attacker label is empty, so FE4's neutral wording and colours are used everywhere.
>
> **Without it:** Every attacker would be shown as the owner and every accepted attack as a "Correct decision", because the app sends the default label "genuine" for every session.

The app already sends a session type and label (`Keydata.kt` , lines 1145–1146), but the bank app never sets them, so every session arrives with the SDK defaults: `enrollment` and `genuine` (`Keydata.kt` , lines 50–51). Trusting them would mark every attacker as the owner. `_truth()` therefore returns `None` unless `EXPLAIN_TRUST_LABELS=true` , which should be set only in a test environment where a test build sets real labels.

DA6. Dashboard backend checks before handing over

> **ON THE DASHBOARD** **Required, not visible** Checks
>
> **Nothing visible. Confirms the APIs return the same numbers as today's file.**

- For one research user whose data is loaded into staging, the three APIs return the same values as today's `work/dashboard_data_risk.json` for the same sessions (allowing for `truth` , `san` and `par` on enrolment, and `fields`).
- `check_session()` passes for every session with a breakdown; no response ever has both `rk` and `rkError` .
- Response times on staging: session under 1 s, user under 2 s uncached, list under 1 s for 30 days.

What the dashboard backend hands over

The frontend gets the same objects the page uses today, so the contract is the existing file `work/dashboard_data_risk.json` :

API	Returns	Same as today's
`explain/user`	One user object	`users[i]` without `tests` and `summary`
`explain/session`	`{user, session}`	One element of `users[i].tests` , plus `fields`
`explain/sessions`	Light rows (+ optional summary)	The fields of `users[i].tests` the table and charts use; `summary` without the owner/attacker split

The static parts (`groups` , `glossary` , `sanSensors`) come from `glossary.py` ; serve them once, for example as a static JSON file, or let the frontend bundle them.

Dashboard frontend team

The page keeps every view, chart and animation; only where its data comes from changes. Today all data is baked into `template_risk.html` at build time. After the change, the page fetches one user and one session at a time from

the dashboard backend's APIs, in the same shape it uses now. File: `explain_dashboard/template_risk.html` (line numbers below are from that file).

FE1. Load data from the APIs instead of the built-in file

ON THE DASHBOARD **Required**

Every page loads live data from the three APIs.

- A "Loading…" state while data arrives, and a clear error message if a page cannot load.

Without it: The page would keep showing the built-in research data.

Before (lines 557–585):

```
<script id="data" type="application/json">/*__DATA__*/null</script>
...
const DATA = JSON.parse(document.getElementById('data').textContent);
const GROUPS = DATA.groups;
const GLOSS = DATA.glossary;
const USERS = DATA.users;
const UMAP = Object.fromEntries(USERS.map(u => [u.id, u]));
...
for (const u of USERS) {
  u.nf = u.features.length;
  ...
}
const ALL_TESTS = USERS.flatMap(u => u.tests);
const JUDGED = ALL_TESTS.filter(s => s.v !== null);
```

After:

```
const API = '/mb/api/v4/explain';
const get = url => fetch(url, { credentials: 'include' }).then(r => {   // sends the
dashboard login cookie
  if (r.status === 401) { location.href = LOGIN_URL; throw new Error('login'); }
  if (!r.ok) throw new Error('HTTP ' + r.status);
  return r.json();
});
const STATIC = await get('explorer_static.json');      // groups, glossary, sanSensors
(FE6)
const DATA = { ...STATIC, users: [] };
const GROUPS = DATA.groups;
const GLOSS = DATA.glossary;
const USERS = DATA.users;                               // filled as users are opened
const UMAP = {};
let ALL_TESTS = [], JUDGED = [];
```

```
function prepareUser(u) {
  // the body of today's `for (const u of USERS)` loop, unchanged
}

async function ensureUser(id) {
  if (UMAP[id]) return UMAP[id];
  const u = await get(`${API}/user?user_id=${encodeURIComponent(id)}`);
  const list = await get(`${API}/sessions?
user_id=${encodeURIComponent(id)}&summary=1`);
  u.tests = list.rows;
  u.summary = list.summary;
  prepareUser(u);
  USERS.push(u); UMAP[id] = u;
  ALL_TESTS = USERS.flatMap(x => x.tests);
  JUDGED = ALL_TESTS.filter(s => s.v !== null);
  return u;
}

async function ensureSession(sessionId) {
  const { user, session } = await get(`${API}/session?
session_id=${encodeURIComponent(sessionId)}`);
  const u = await ensureUser(user);
  let i = u.tests.findIndex(s => s.id === session.id);   // swap the light row for the
full session
  if (i < 0) { i = u.tests.length; u.tests.push(session); } else u.tests[i] = session;
  session.i = i; session.u = u;
  return [u, session];
}
```

The script tag becomes `<script type="module">` so that top-level `await` works. `ALL_TESTS` and `JUDGED` become `let`, because they grow as users are opened.

FE2. Route by session ID, and render after loading

ON THE DASHBOARD **Required**

Links and pickers work with production session IDs.

- Session links as `#s-<session_id>`, so a link from the dashboard opens the right session.
- The person picker, the “compare with” picker on the fingerprint, and the worked example on the How page.

Without it: Links would point at list positions that do not exist in production.

Production session IDs are unique, so a session link no longer needs the user name and list position.

Before (line 678, and the first branch of `render()`, line 3003):

```
const sessHref = s => `#s-${s.u.id}-${s.i}`;
...
if ((m = hash.match(/^s-(.+)-(\d+)$/)) && UMAP[m[1]] && UMAP[m[1]].tests[+m[2]]) {
  const u = UMAP[m[1]];
  curSession = u.tests[+m[2]];
  html = viewSession(u, curSession); curUser = u.id;
}
```

After:

```
const sessHref = s => `#s-${encodeURIComponent(s.id)}`;
...
async function render() {
  ...                                                    // unchanged lines above
  app.innerHTML = '<p class="muted">Loading…</p>';
  try {
    if ((m = hash.match(/^s-(.+)$/))) {
      const [u, s] = await ensureSession(m[1]);
      curSession = s;
      html = viewSession(u, s); curUser = u.id;
    } else if ((m = hash.match(/^u-(.+)$/))) {
      html = viewUser(await ensureUser(m[1])); curUser = m[1];
    } else ...                                           // how, features, method:
unchanged
  } catch (e) {
    html = `<p class="err">Could not load this page (${esc(e.message)}).</p>`;
  }
  ...                                                    // unchanged lines below
}
```

Also:

- The "Person" picker (line 2958) is filled from `USERS`, which now starts empty. Fill it from the existing dashboard user list, or drop it and open people from the dashboard.
- The session page's "compare with" picker (line 1886) lists other sessions from the light rows. When one is picked, call `ensureSession()` for it before redrawing, because a light row has no measurement values.
- The How and Method pages (lines 2399–2400 and 2887–2934) pick their worked example from `ALL_TESTS`. Point them at one fixed example session ID from config, loaded with `ensureSession()`.
- `render()` now waits for the network, so two quick hash changes can finish out of order. Keep a counter: each call takes the next number and drops its result if a newer call has started.
- The page has no inline event handlers, so the switch to `<script type="module">` needs no other change; the start-up `render()` call (line 3097) stays.

FE3. Keep, unchanged

ON THE DASHBOARD **Required** No work

Everything the demo shows stays: the risk section, the evidence strips, the risk card animation, the definitions page and its 50 animations, the glow and back pill, the Animations switch.

These read the same fields as today, so no change: the risk section (gauge, tug of war, merged pull/push evidence with its two strips, group waterfall, S-curve, exact arithmetic), the risk card with its squeeze animation, the fingerprint, the counterfactual, the reference sessions, the replay, the definitions page with its 50 animations, the glow on arrival, the back pill, and the Animations switch.

FE4. Production has no owner/attacker labels

ON THE DASHBOARD **Changes the page** Hidden or reworded in production

What disappears or changes when there are no owner/attacker labels.

- Banner: no “Who really used the phone” line and no “Wrong: attacker accepted” badge.
- **Why**: no “Correct decision” / “Wrong decision … false acceptance” line.
- Charts: one neutral colour (or colour by verdict) instead of owner blue and attacker orange.
- Person page: no attackers table and no “What gives attackers away” section.
- Overview: accepted / rejected / no-verdict counts instead of owner and attack results.

Without it: With the labels empty, today’s page code treats every session as an attack: it would name the account’s own user as the attacker and mark every acceptance “Wrong: attacker accepted”.

Why

Accepted as genuine. Risk score **33.55**, below the line at 50. In the model's own terms, this session differed from pritam's reference sessions by only **1.67** difference points, within the **1.86** allowed. It used 90% of the allowed difference, leaving 10% to spare.

32 of the 50 measurements were inside pritam's usual range, and 6 measurements fell outside anything seen during enrolment. The closest of pritam's reference sessions (enrolment session 5, recorded on 31 Aug, 13:13 IST) carried 16% of the support for this session.

The largest difference was in typing rhythm (34% of the total). The biggest single one: **Re-entering the account number: typing speed** was 3.08 keys/s, faster than in all 50 of pritam's enrolment sessions (highest there: 3.05 keys/s); pritam's typical value is 1.41 keys/s.

Wrong decision. This was Nikitha18, not pritam, so this is a false acceptance. Nikitha18's behaviour happened to sit close to pritam's on most measurements.

Example from today's Verdict Explorer (research data): the Why section. The red last line needs a real label, so it is dropped in production.

What gives attackers away on pritam's phone

Across the 64 attacks the model rejected on pritam's account, the attackers' behaviour differed from pritam's by **5.32** points on average, against **1.40** for pritam's own later sessions. pritam's allowed difference is about 1.80 points, so a typical attack used 195% of it and a typical pritam session 72%.

Every behaviour group contributes, but not equally. Attackers stand out most in **grip & hand motion**, where they showed 7.3× the difference pritam's own sessions did, then typing rhythm (4.4×), touch rhythm & placement (1.8×), tap impact (1.6×).

The single measurements that separate attackers from pritam best are **fast tilt wobble (12 to 25 hz)**, **tremor share in tipping (8 to 12 hz)**, **fast jitter share in tipping**. For example, in fast tilt wobble (12 to 25 hz), attackers' median was 0.00241 deg², against 0.00615 deg² in pritam's later sessions and 0.00389 deg² during enrolment. 31 of 74 attacks fell outside pritam's usual range (0.00175 deg² to 0.0107 deg²), compared with 9 of 25 of pritam's own later sessions.

1 · Which behaviour groups separate attackers from pritam

Average difference points from each behaviour group, for the attacks the model rejected and for pritam's own later sessions. The larger the gap between the two bars, the more that behaviour gives attackers away.



Example from today's Verdict Explorer (research data): this section compares attackers with the owner, so it is hidden in production.



Results by person

Click a row to open that person's sessions.

Person	Enrolment	Owner sessions accepted	Owner sessions rejected	Attacks rejected	Attacks accepted	No verdict
Parthish one phone	50	19 / 24	5	80 / 80	0	0
Teja one phone	50	15 / 24	9	79 / 80	1	0
akshaya 2 phones	50	18 / 23	5	79 / 80	1	0
pritam one phone	50	23 / 25	2	64 / 74	10	1
ram one phone	50	22 / 24	2	80 / 80	0	0
reddy one phone	50	47 / 50	3	75 / 75	0	0
rohity one phone	50	23 / 24	1	76 / 80	4	0
surya one phone	50	40 / 50	10	75 / 75	0	0

Example from today's Verdict Explorer (research data): results by person. Owner and attack counts become accepted and rejected counts.

`truth` is `null` in production (DA5). The page assumes every session is either the owner's (`'g'`) or an attack (`'i'`), and in about 30 places it colours, words or counts sessions that way. Add one helper and use it at each place:

```
const hasTruth = s => s.truth === 'g' || s.truth === 'i';
```

Lines	Today	When `hasTruth(s)` is false
579–580	Splits sessions into owner and attack lists	Both lists empty; use `u.tests`
670–677	"X (owner)" / "Y (attacker)", right/wrong decision, `isWrong`	"X's account"; no right/wrong; `isWrong` is false
812, 830–844	Why text ends "Correct decision …" or "Wrong decision … false acceptance"	End with the verdict and risk score only
914, 998, 1169, 1204, 1345	Tooltips and dots coloured owner or attacker	One neutral colour; colour by verdict instead
1522–1544, 1555	Overview: owner/attack counts, false accepts and false rejects	Accepted, rejected and no-verdict counts only
1565–1631	Person page: owner vs attack tallies, attackers table	Accepted/rejected tallies; hide the attackers table
1672–1800	"What gives attackers away" section and its fingerprint picker	Hide the section
1886	"Compare with" picker grouped into later sessions and attacks	One group: other sessions

When a labelled test environment sets `EXPLAIN_TRUST_LABELS=true`, `truth` is filled and every view works as it does today.

FE5. Sessions without a breakdown

ON THE DASHBOARD **Changes the page** A short note instead of the breakdown

Sessions with a verdict but no breakdown show one line instead of the risk section.

- "The breakdown is not available for this session." under the verdict and risk score.

Without it: The risk section would break on sessions scored before the release (if DB3 is skipped) or on a failed check.

Some sessions have a verdict but no breakdown: scored before ML4 and not backfilled, or a breakdown that failed its checks (`rkError`). Where the page draws the risk section, check `s.rk` (not only `s.v !== null`). Without it, show the verdict and risk score with one line: "The breakdown is not available for this session." Sessions with no verdict keep today's sanitizer message.

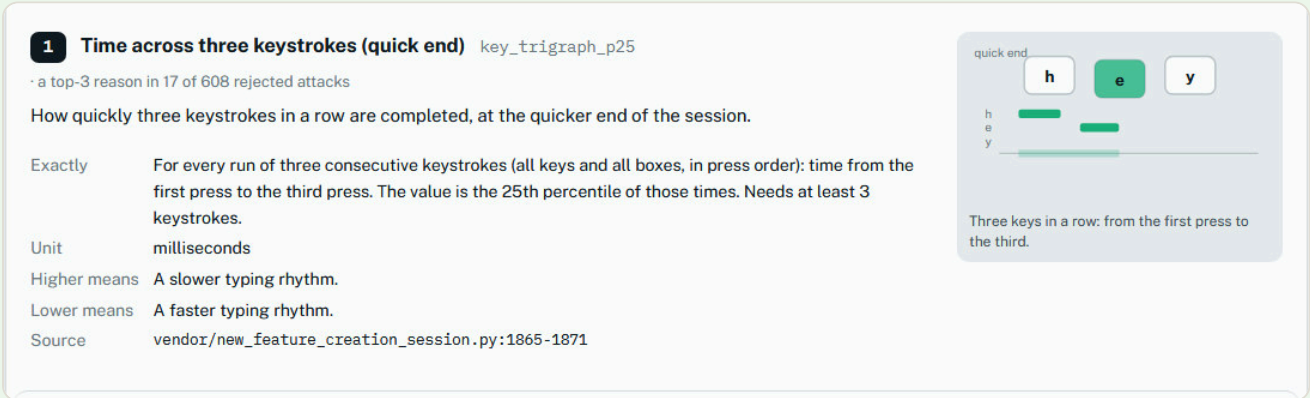
FE6. Static parts and replay fields

ON THE DASHBOARD **Required**

The definitions and the replay's field names.

- The **definitions page**: names, explanations, groups and the 50 animations (from `explorer_static.json`).
- The field names in the **raw recording** ("which box was typed in").

Without it: Measurement names, definitions and groups would be missing everywhere.



Example from today's Verdict Explorer (research data): a definitions entry with its animation; all of this text is static.

- `groups`, `glossary` and `sanSensors` are the same for every user. Generate `explorer_static.json` once from `glossary.py` and serve it next to the page.
- A session now carries its own `fields` list for its key replay. Decode a session's replay keys with `s.fields || s.u.fields`.

FE7. Hosting

ON THE DASHBOARD **Required**

How people reach the page.

- Each row of the dashboard's session logs links straight to that session's explanation.

Without it: The page would exist but nobody could open it from the dashboard.

- Serve the page from the dashboard's own origin, so the login cookie reaches the APIs; link each row of the dashboard's session logs to `<page>#s-<session_id>`.
- The 6 MB offline file and the build scripts are no longer needed in production. Keep them for the demo.
- The page loads fonts from Google Fonts. If the dashboard's security policy blocks outside hosts, serve the three fonts locally.

Frontend checks before release

- `test_render_risk.js`, pointed at API responses for the staging users instead of the built-in file: 0 problems, including every printed equation multiplying out.
- `flow.js` click path on staging: risk card, strips, definition links with the glow and back pill, motion switch, next session; no page errors.
- Every page with `truth = null`: no "owner", "attacker", "correct" or "wrong" wording anywhere.
- Phone width 390 px: no sideways scrolling on the session, features and how pages.

What changes, and what production cannot show

Every number on the page comes from the production decision itself, computed in the same request as the verdict. Nothing the bank's users see changes.

	Today	After these changes
Where the numbers come from	A research copy of the data, re-scored offline on another PC	The production verdict, computed in the same request
A new session appears on the page	After a full rebuild of a 6 MB file	As soon as it is scored
Exactness	Recomputed later, then checked against production	Same request, same model, same feature values; checked again on every page load

	Today	After these changes
Training values (ranges, typical values, fingerprint)	The research training CSV	Inside each model file (ML1), or its sidecar (ML6)
Verdicts, scores, thresholds, risk scores	–	Unchanged
Mobile app and SDK	–	Unchanged
Oracle	–	One new column, about 5 KB per scored session
Predict reply	–	One new field, `explain`, about 4–5 KB; can be switched off with `ML_EXPLAIN=false`

What production cannot show: whether a session was really the owner or an attacker. The research data had real labels, for example "reddy using pritam's account". In production the app sends only the SDK's default labels, so the owner/attacker views, "correct" and "wrong" decisions, false accepts, false rejects and FAR/FRR are hidden (FE4). They come back unchanged in any test environment where a test build sets real labels.

Open questions to settle before starting

- [ ] **Uncertified feature list.** v8 is marked "ceiling probe, viable = false" in `selections.json`, and its models are served as uncertified. The page already shows the model's warning. Should it stay visible in production? (Product owner)
- [ ] **Who may see it.** The page shows raw behaviour measurements and session replays. Which dashboard roles get the three APIs, and does this need a data-protection sign-off? (Product owner, security)
- [ ] **Labelled test flow.** Do we want the owner/attacker views back in a test environment? That needs a test build that calls the SDK's `setSessionLabel()` and `setSessionType()`, which no team on this list owns. (App/SDK team)
- [ ] **Model storage in production.** Which ECS volumes hold `/app/saved_models_v8`, `/app/templates_v8` and `/app/jobs_v8` today (ML7)? (ML team, DevOps)
- [ ] **Retention.** Should `EXPLAIN_JSON` follow the same retention as the rest of `NAV_SESSION_SCORES`? (DB team)
- [ ] **Latency budget.** Is the predict time measured in the ML checks acceptable with `ML_EXPLAIN` on? (ML team, backend team)
- [ ] **Known false rejections.** The false-positive flag is written to `patterngame_svm_user_signin_info`. Should the page show it on NAV sessions, as a reviewer's "this was really the owner" mark? (Backend team)